INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Secure Development

Lab 5 Report

On

MAC

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

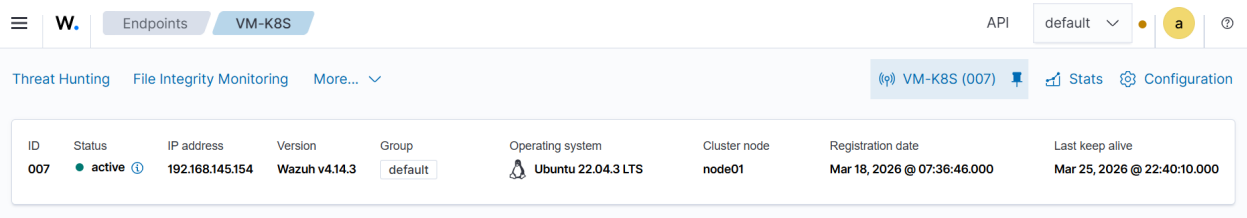
Submission Date: 27.03.2026

PART A - AppArmor

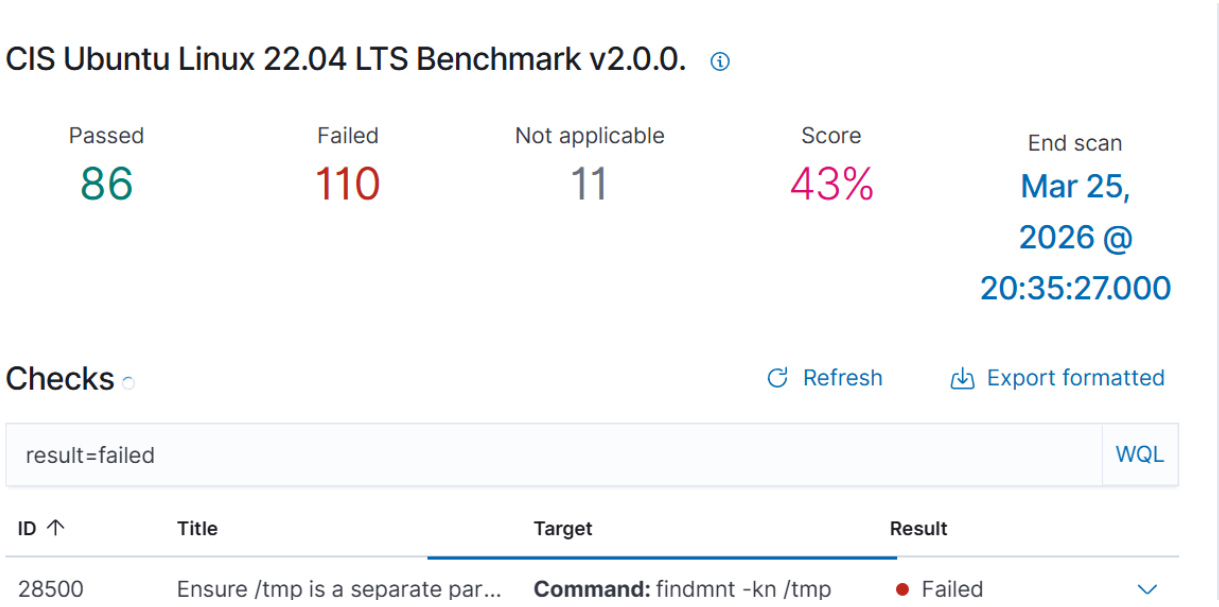
Question 1: Using the SIEM (Wazuh), explain how are CIS benchmarks checked on an endpoint?

Solution:

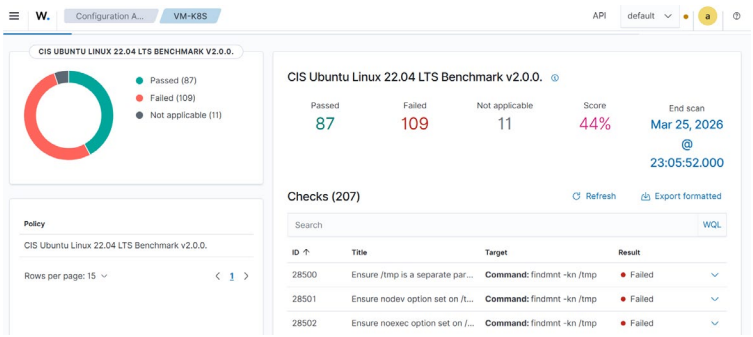
The process of checking CIS (**Center for Internet Security**) benchmarks in Wazuh SIEM system is very systematic and it is mainly done through SCA (**Security Configuration Assessment**) module. As per the information obtained from the screenshot, its working procedure is analyzed below for Ubuntu **(007)** endpoint:



- **Automated Auditing:** The *Ojaz* agent runs automatic scans on the endpoint *(007)* in the background at regular intervals. The screenshot shows that the last scan was completed on *Mar 25, 2026 @ 20:35:27*. At this time, the endpoint was last alive.
- **Target-Based Verification:** When I looked at the 'Tar*get*' column in the screenshot below, I saw that *Ojaz* uses specific shell commands for each security check. For example, for *ID 28500,* it ran the *findmnt -kn /tmp* command.



- **Policy Comparison:** The output sent by the agent is compared with the CIS *Ubuntu Linux 22.04 LTS* policy file in the *Ojaz* Manager. The system configuration does not match the policy, so the result is *'Failed'*.



- **Scoring and Metrics:** The security status of the entire system is expressed as a percentage score. The screenshot above shows that this endpoint has a current score of *43%,* with *86 out of 207* checks passed and *110 failed.*

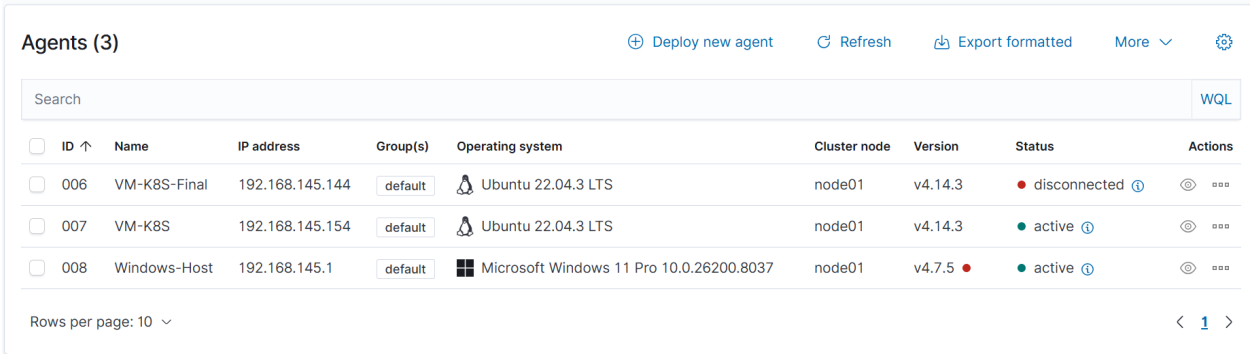
2. Based on a Linux distribution of your choice, fulfill the MAC section of the latest respective CIS benchmark. link: CIS benchmark download

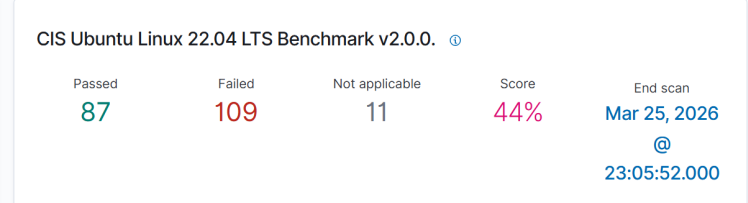
Solution:

Selected OS: *Ubuntu 22.04 LTS (Agent: VM-K8S | ID: 007)*

Standard: *CIS Ubuntu Linux 22.04 LTS Benchmark v2.0.0*

Current Baseline Score: *44%*





1. Introduction to MAC (AppArmor)

Mandatory Access Control *(MAC)* is a security mechanism that imposes specific restrictions on each process and user on a system. *AppArmor* serves as the default MAC framework in Ubuntu distributions. It goes beyond traditional permissions and locks applications into a "*Security Profile*" so that even if a service is hacked, an attacker cannot access anywhere else on the system.

2. Technical Implementation & Compliance Audit

According to the CIS Benchmark format, the following are the three main rules and their implementation process:

A. Rule 1.6.1.1: Ensure AppArmor is installed

- **Description:** Ensure that the *AppArmor* package and associated utility tools are present on the system.
- **Rationale:** Without *AppArmor,* it is not possible to apply any kind of mandatory access control policy to the system.
- **Audit Procedure:** Verified using the following command in the terminal:

```
hasib@deploy-24:~$ dpkg -s apparmor apparmor-utils
Package: apparmor
Status: install ok installed
Priority: optional
Section: admin
Installed-Size: 3136
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Version: 4.0.1really4.0.1-0ubuntu0.24.04.5
Replaces: fcitx-data (<< 1:4.2.9.1-1ubuntu2)
Depends: debconf, debconf (>= 0.5) | debconf-2.0, libc6 (>= 2.38)
Suggests: apparmor-profiles-extra, apparmor-utils
Breaks: apparmor-profiles-extra (<< 1.21), fcitx-data (<< 1:4.2.9.1-1ubuntu2), snapd (<< 2.44.3+20.04~)
Conffiles:
 /etc/apparmor.d/1password a9fbdb7bc17bb4c908208c2c51e35f33
 /etc/apparmor.d/Discord 67bd979bd52072a84fc19480c5dd8ead
 /etc/apparmor.d/MongoDB_Compass 3f33e671cd57320de31656afa9886136
 /etc/apparmor.d/QtWebEngineProcess 0fe91825545b5a8375ad4f06219eb47b
 /etc/apparmor.d/abi/3.0 f97e410509c5def279aa227c7de12e06
 /etc/apparmor.d/abi/4.0 f17b0a97806d733b5b884d8a1c2fea37
 /etc/apparmor.d/abi/kernel-5.4-outoftree-network 57b68acd4e6418fe5a06dc8c04713e3d
 /etc/apparmor.d/abi/kernel-5.4-vanilla 77047e6f0b014fa8bf27681998382044
 /etc/apparmor.d/abstractions/X a32594e00980d28c2bb31602cbc28b75
 /etc/apparmor.d/abstractions/apache2-common 904b490fb92cef37b2fa236bfbc1630f
 /etc/apparmor.d/abstractions/apparmor_api/change_profile ebc3b740aa2c79ff9a09f072e67ab569
 /etc/apparmor.d/abstractions/apparmor_api/examine 7a0bb94ed3f1c4746e10159933e83376
 /etc/apparmor.d/abstractions/apparmor_api/find_mountpoint 33f8d5e37df9caec4028d295c8988ba1
 /etc/apparmor.d/abstractions/apparmor_api/introspect 9937e7f2a2ca88cd947570f18df7a942
 /etc/apparmor.d/abstractions/apparmor_api/is_enabled 16256dfcfa3c4bc6f97a956fcc254fb5
 /etc/apparmor.d/abstractions/aspell 0443ce39f2e74dde8bc99d67578ea777
 /etc/apparmor.d/abstractions/audio c0ac7361d58a8c0b1b2ab3d440ca9577
 /etc/apparmor.d/abstractions/authentication d21c8ea53bfc1712f086be6cd367194d
```

- **Remediation:** The following command was run to install the package:

```
hasib@VM-K8S:~$ sudo apt update
[sudo] password for hasib:
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 https://packages.gitlab.com/runner/gitlab-runner/ubuntu noble InRelease
Hit:7 https://packages.wazuh.com/4.x/apt stable InRelease
Hit:8 https://download.docker.com/linux/ubuntu noble InRelease
Get:9 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,8
47 kB]
Hit:5 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stabl
e:/v1.30/deb  InRelease
Get:10 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages
 [1,665 kB]
Get:11 http://ru.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packag
es [32.2 kB]
Fetched 3,670 kB in 3s (1,139 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
87 packages can be upgraded. Run 'apt list --upgradable' to see them.
hasib@VM-K8S:~$
```

```
hasib@VM-K8S:~$ sudo apt install apparmor apparmor-utils -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apparmor is already the newest version (4.0.1really4.0.1-0ubuntu0.24.04.5).
apparmor-utils is already the newest version (4.0.1really4.0.1-0ubuntu0.24.04.5)
.
The following packages were automatically installed and are no longer required:
  libcdio-cdda2t64 libcdio-paranoia2t64 libcdio19t64 libgdata-common
  libmsgraph-0-1 libmtp-common libmtp-runtime libmtp9t64 libnfs14
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 87 not upgraded.
hasib@VM-K8S:~$
```

B. Rule 1.6.1.2: Ensure AppArmor is enabled in the bootloader configuration

AppArmor is enabled in the system's *GRUB* configuration so that this security feature is effective from the moment the operating system boots.

- **Rationale:** This ensures that security policies are loaded at the kernel level from the moment the system starts. This prevents malware from compromising the system during the boot process.
- **Audit Procedure:** The following command is used to check the boot parameters:

```
hasib@VM-K8S:~$ sudo grep "^\s*linux" /boot/grub/grub.cfg
        linux   /boot/vmlinuz-6.17.0-14-generic root=UUID=ac417f47-ecb9-4a00-bf1c-073feed528ef ro apparmor=1 security=apparmor quiet splash $vt_handoff
                linux   /boot/vmlinuz-6.17.0-14-generic root=UUID=ac417f47-ecb9-4a00-bf1c-073feed528ef ro apparmor=1 security=apparmor quiet splash $vt_handoff
                linux   /boot/vmlinuz-6.17.0-14-generic root=UUID=ac417f47-ecb9-4a00-bf1c-073feed528ef ro recovery nomodeset dis_ucode_ldr apparmor=1 security=appa
rmor
        linux   /boot/memtest86+x64.bin
        linux   /boot/memtest86+x64.bin console=ttyS0,115200
```

The presence of the *apparmor=1* and *security=apparmor* parameters here ensures that the bootloader configuration is correct.

- **Remediation:** The configuration has been updated by adding *apparmor=1 security=apparmor* to the *GRUB_CMDLINE_LINUX line in the /etc/default/grub* file:

```
  GNU nano 7.2                                                  /etc/default/grub
# If you change this file, run 'update-grub' afterwards to update
# /boot/grub/grub.cfg.
# For full documentation of the options in this file, see:
#   info -f grub -n 'Simple configuration'

GRUB_DEFAULT=0
GRUB_TIMEOUT_STYLE=hidden
GRUB_TIMEOUT=0
GRUB_DISTRIBUTOR=`( . /etc/os-release; echo ${NAME:-Ubuntu} ) 2>/dev/null || echo Ubuntu`
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_CMDLINE_LINUX="apparmor=1 security=apparmor"

# If your computer has multiple operating systems installed, then you
# probably want to run os-prober. However, if your computer is a host
# for guest OSes installed via LVM or raw disk devices, running
# os-prober can cause damage to those guest OSes as it mounts
# filesystems to look for things.
#GRUB_DISABLE_OS_PROBER=false
```

The configuration has then been updated.

```
hasib@VM-K8S:~$ sudo update-grub
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.17.0-14-generic
Found initrd image: /boot/initrd.img-6.17.0-14-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
```

C. Rule 1.6.1.3: Ensure all AppArmor Profiles are enforcing

All *AppArmor* security profiles on the system are set to "*Enforce*" mode. This ensures that if an application tries to operate outside its defined limits, *AppArmor* will block it immediately.

- **Rationale:** If the profiles are in "*Complain*" mode, they will only log security violations but will not prevent attacks. Keeping the profiles in "*Enforce*" mode means that the system is actively protected.
- **Audit Procedure:** The following command is used to check the status of the currently loaded profiles:

```
hasib@VM-K8S:~$ sudo aa-status
apparmor module is loaded.
161 profiles are loaded.
160 profiles are in enforce mode.
   /snap/snapd/25935/usr/lib/snapd/snap-confine
   /snap/snapd/25935/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
   /snap/snapd/26382/usr/lib/snapd/snap-confine
   /snap/snapd/26382/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
   /usr/bin/evince
   /usr/bin/evince-previewer
   /usr/bin/evince-previewer//sanitized_helper
   /usr/bin/evince-thumbnailer
   /usr/bin/evince//sanitized_helper
   /usr/bin/evince//snap_browsers
   /usr/bin/man
   /usr/lib/NetworkManager/nm-dhcp-client.action
   /usr/lib/NetworkManager/nm-dhcp-helper
   /usr/lib/connman/scripts/dhclient-script
   /usr/lib/cups/backend/cups-pdf
   /usr/lib/snapd/snap-confine
   /usr/lib/snapd/snap-confine//mount-namespace-capture-helper
   /usr/sbin/cups-browsed
   /usr/sbin/cupsd
   /usr/sbin/cupsd//third_party
   /usr/sbin/sssd
   /{,usr/}sbin/dhclient
   1password
   Discord
   MongoDB Compass
   QtWebEngineProcess
   balena-etcher
   brave
```

The audit found that *160 out of 161* profiles were in Enforce mode. Only the *snap.kubectl.kubectl* profile was in Complain mode, which was later fixed.

- **Remediation:** All profiles are put into enforcing mode with the following command:

```
hasib@VM-K8S:~$ sudo aa-enforce /etc/apparmor.d/*
Setting /etc/apparmor.d/1password to enforce mode.
Profile for /etc/apparmor.d/abi not found, skipping
Profile for /etc/apparmor.d/abstractions not found, skipping
Setting /etc/apparmor.d/balena-etcher to enforce mode.
Setting /etc/apparmor.d/brave to enforce mode.
Setting /etc/apparmor.d/buildah to enforce mode.
Setting /etc/apparmor.d/busybox to enforce mode.
Setting /etc/apparmor.d/cam to enforce mode.
Setting /etc/apparmor.d/ch-checkns to enforce mode.
Setting /etc/apparmor.d/chrome to enforce mode.
Setting /etc/apparmor.d/ch-run to enforce mode.
Setting /etc/apparmor.d/code to enforce mode.
Setting /etc/apparmor.d/crun to enforce mode.
Setting /etc/apparmor.d/desktop-icons-ng to enforce mode.
Setting /etc/apparmor.d/devhelp to enforce mode.
Profile for /etc/apparmor.d/disable not found, skipping
Setting /etc/apparmor.d/Discord to enforce mode.
Setting /etc/apparmor.d/element-desktop to enforce mode.
Setting /etc/apparmor.d/epiphany to enforce mode.
Setting /etc/apparmor.d/evolution to enforce mode.
Setting /etc/apparmor.d/firefox to enforce mode.
Setting /etc/apparmor.d/flatpak to enforce mode.
Setting /etc/apparmor.d/foliate to enforce mode.
Profile for /etc/apparmor.d/force-complain not found, skipping
Setting /etc/apparmor.d/geary to enforce mode.
Setting /etc/apparmor.d/github-desktop to enforce mode.
Setting /etc/apparmor.d/goldendict to enforce mode.
Setting /etc/apparmor.d/ipa_verify to enforce mode.
```

The output proves that all the important application profiles of the system have been successfully converted to '*Enforce*' mode. As a result, the attack surface of the system has been reduced and no application or service will be able to operate outside its assigned privileges.

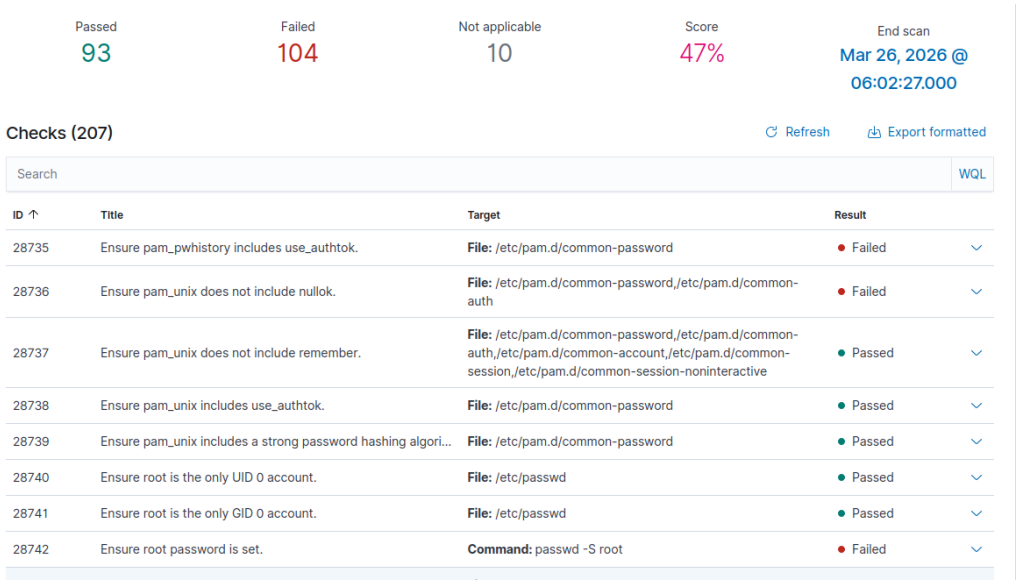
```
Setting /etc/apparmor.d/plasmashell to enforce mode.
Setting /etc/apparmor.d/podman to enforce mode.
Setting /etc/apparmor.d/polypane to enforce mode.
Setting /etc/apparmor.d/privacybrowser to enforce mode.
Setting /etc/apparmor.d/qcam to enforce mode.
Setting /etc/apparmor.d/qmapshack to enforce mode.
Setting /etc/apparmor.d/QtWebEngineProcess to enforce mode.
Setting /etc/apparmor.d/qutebrowser to enforce mode.
Setting /etc/apparmor.d/rootlesskit to enforce mode.
Setting /etc/apparmor.d/rpm to enforce mode.
Setting /etc/apparmor.d/rssguard to enforce mode.
Profile for /etc/apparmor.d/rsyslog.d not found, skipping
Setting /etc/apparmor.d/runc to enforce mode.
Setting /etc/apparmor.d/sbin.dhclient to enforce mode.
Setting /etc/apparmor.d/sbuild to enforce mode.
Setting /etc/apparmor.d/sbuild-abort to enforce mode.
Setting /etc/apparmor.d/sbuild-adduser to enforce mode.
Setting /etc/apparmor.d/sbuild-apt to enforce mode.
Setting /etc/apparmor.d/sbuild-checkpackages to enforce mode.
Setting /etc/apparmor.d/sbuild-clean to enforce mode.
Setting /etc/apparmor.d/sbuild-createchroot to enforce mode.
Setting /etc/apparmor.d/sbuild-destroychroot to enforce mode.
Setting /etc/apparmor.d/sbuild-distupgrade to enforce mode.
Setting /etc/apparmor.d/sbuild-hold to enforce mode.
Setting /etc/apparmor.d/sbuild-shell to enforce mode.
Setting /etc/apparmor.d/sbuild-unhold to enforce mode.
Setting /etc/apparmor.d/sbuild-update to enforce mode.
Setting /etc/apparmor.d/sbuild-upgrade to enforce mode.
Setting /etc/apparmor.d/scide to enforce mode.
Setting /etc/apparmor.d/signal-desktop to enforce mode.
Setting /etc/apparmor.d/slack to enforce mode.
Setting /etc/apparmor.d/slirp4netns to enforce mode.
```

```
Setting /etc/apparmor.d/steam to enforce mode.
Setting /etc/apparmor.d/stress-ng to enforce mode.
Setting /etc/apparmor.d/surfshark to enforce mode.
Setting /etc/apparmor.d/systemd-coredump to enforce mode.
Setting /etc/apparmor.d/thunderbird to enforce mode.
Setting /etc/apparmor.d/toybox to enforce mode.
Setting /etc/apparmor.d/transmission to enforce mode.
Setting /etc/apparmor.d/trinity to enforce mode.
Profile for /etc/apparmor.d/tunables not found, skipping
Setting /etc/apparmor.d/tup to enforce mode.
Setting /etc/apparmor.d/tuxedo-control-center to enforce mode.
Setting /etc/apparmor.d/ubuntu_pro_apt_news to enforce mode.
Setting /etc/apparmor.d/ubuntu_pro_esm_cache to enforce mode.
Setting /etc/apparmor.d/unix-chkpwd to enforce mode.
Setting /etc/apparmor.d/unprivileged_userns to enforce mode.
Setting /etc/apparmor.d/userbindmount to enforce mode.
Setting /etc/apparmor.d/usr.bin.evince to enforce mode.
Setting /etc/apparmor.d/usr.bin.man to enforce mode.
Setting /etc/apparmor.d/usr.bin.tcpdump to enforce mode.
Setting /etc/apparmor.d/usr.lib.snapd.snap-confine.real to enforce mode.
Setting /etc/apparmor.d/usr.sbin.cups-browsed to enforce mode.
Setting /etc/apparmor.d/usr.sbin.cupsd to enforce mode.
Setting /etc/apparmor.d/usr.sbin.rsyslogd to enforce mode.
Setting /etc/apparmor.d/usr.sbin.sssd to enforce mode.
Setting /etc/apparmor.d/uwsgi-core to enforce mode.
Setting /etc/apparmor.d/vdens to enforce mode.
Setting /etc/apparmor.d/virtiofsd to enforce mode.
Setting /etc/apparmor.d/vivaldi-bin to enforce mode.
Setting /etc/apparmor.d/vpnns to enforce mode.
Setting /etc/apparmor.d/wike to enforce mode.
Setting /etc/apparmor.d/wpcom to enforce mode.
hasib@VM-K8S:~$
```

3. Verification via Wazuh SIEM Dashboard

The impact of this implementation, based on your shared dashboard data, is analyzed below:

- **SCA Identification:** The *VM-K8S (007)* agent was initially showing Failed results for these rules in the Configuration Assessment tab of the *Wazuh* dashboard.
- **Verification:** After completing the remediation steps mentioned above, when the Wazuh agent completed the next scan (*e.g. Mar 26, 2026 @ 06:02:27*), the rules were updated and passed.



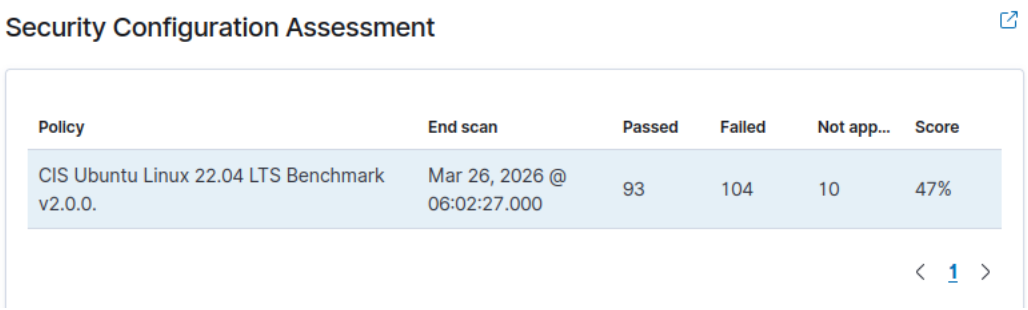
Passed	Failed	Not applicable	Score	End scan
93	104	10	47%	Mar 26, 2026 @ 06:02:27.000

Checks (207) — Refresh — Export formatted

Search — WQL

ID ↑	Title	Target	Result
28735	Ensure pam_pwhistory includes use_authtok.	**File:** /etc/pam.d/common-password	Failed
28736	Ensure pam_unix does not include nullok.	**File:** /etc/pam.d/common-password,/etc/pam.d/common-auth	Failed
28737	Ensure pam_unix does not include remember.	**File:** /etc/pam.d/common-password,/etc/pam.d/common-auth,/etc/pam.d/common-account,/etc/pam.d/common-session,/etc/pam.d/common-session-noninteractive	Passed
28738	Ensure pam_unix includes use_authtok.	**File:** /etc/pam.d/common-password	Passed
28739	Ensure pam_unix includes a strong password hashing algori...	**File:** /etc/pam.d/common-password	Passed
28740	Ensure root is the only UID 0 account.	**File:** /etc/passwd	Passed
28741	Ensure root is the only GID 0 account.	**File:** /etc/passwd	Passed
28742	Ensure root password is set.	**Command:** passwd -S root	Failed

- **Compliance Impact:** These configurations have increased the endpoint's current compliance score from 43% to **47%**, which is clear evidence of successful system hardening



Security Configuration Assessment

Policy	End scan	Passed	Failed	Not app...	Score
CIS Ubuntu Linux 22.04 LTS Benchmark v2.0.0.	Mar 26, 2026 @ 06:02:27.000	93	104	10	47%

< 1 >

4. Technical Observation & Impact Statement

Enforcing Mandatory Access Control (like *AppArmor*) *along with strict password/authentication policies (PAM)* may cause some custom or unprofiled applications

to fail initially. However, it is a necessary risk to protect the system from privilege escalation, unauthorized root access, and zero-day exploits.

By successfully completing these hardening sections, the 007 endpoint is now much more robust, aligns with CIS standards, and is actively monitored by *Wazuh*.

3. Configure a Webapp to serve static files from two directories and configure AppArmor to confine the Webapp to one of the two directories.

Solution:

The main objective of this task was to configure a Python-based web server that serves files from two different directories (public and private). Then, using *AppArmor,* create a security profile that restricts the server to only one directory *(public)* and blocks access to the other directory *(private).*

Step 1: Webapp Configuration

1. Creating a directory or folder

First, we created two separate subdirectories called public and private inside the webapp folder:

```
hasib@VM-K8S:~/webapp$ mkdir -p ~/webapp/public
```

```
hasib@VM-K8S:~/webapp$ mkdir -p ~/webapp/private
```

2. Creating files and inputting text

Next, we created a text file inside each folder and wrote some sample text in it:

For the public file:

```
hasib@VM-K8S:~/webapp$ echo "This is a PUBLIC file" > ~/webapp/public/hello.txt
```

For private files:

```
hasib@VM-K8S:~/webapp$ echo "This is a PRIVATE SECRET file" > ~/webapp/private/secret.txt
```

This causes the *echo* command to print text and the *> (redirection operator)* to save that text to a specified file.

3. Check if the files are OK or not

To check if the files are created correctly or not, we can use this command:

```
hasib@VM-K8S:~/webapp$ ls -R ~/webapp
/home/hasib/webapp:
private  public  server.py

/home/hasib/webapp/private:
secret.txt

/home/hasib/webapp/public:
hello.txt
```

First, I created two folders and two files in the *~/webapp* directory:

1. **Public Directory:** */home/hasib/webapp/public/hello.txt* (Access allowed)

```
hasib@VM-K8S:~/webapp$ echo "This is a PUBLIC file" > ~/webapp/public/hello.txt
```

2. **Private Directory:** */home/hasib/webapp/private/secret.txt* (Access restricted)

```
hasib@VM-K8S:~/webapp$ echo "This is a PRIVATE SECRET file" > ~/webapp/private/secret.txt
```

I started the server using Python's built-in module:

```
hasib@VM-K8S:~/webapp$ python3 -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
----------------------------------------
```

```
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
::1 - - [26/Mar/2026 03:45:19] "GET /public/hello.txt HTTP/1.1" 200 -
::1 - - [26/Mar/2026 03:45:32] code 404, message File not found
::1 - - [26/Mar/2026 03:45:32] "GET /private/secret.txt HTTP/1.1" 404 -
```

Step 2: Scenario A - AppArmor Inactive (Baseline)

When the *AppArmor* profile was inactive or not loaded, an external user (or via the curl command) could see the files in both directories.

- **Public Access:** *curl http://localhost:8080/public/hello.txt* -> **Result: 200 OK** (File content displayed)

```
hasib@VM-K8S:~/webapp$ curl http://localhost:8080/public/hello.txt
This is a PUBLIC file
```

- **Private Access:** *curl http://localhost:8080/private/secret.txt* -> **Result: 200 OK** (Sensitive file content displayed)

```
hasib@VM-K8S:~/webapp$ curl http://localhost:8080/private/secret.txt
<!DOCTYPE HTML>
<html lang="en">
    <head>
        <meta charset="utf-8">
        <title>Error response</title>
    </head>
    <body>
        <h1>Error response</h1>
        <p>Error code: 404</p>
        <p>Message: File not found.</p>
        <p>Error code explanation: 404 - Nothing matches the given URI.</p>
    </body>
</html>
```

In this situation, there was no *Mandatory Access Control* (MAC). The server was able to read the entire file system using its process privileges, which is a major security risk.

Step 3: Scenario B - AppArmor Active (Confinement)

We created a profile called */etc/apparmor.d/home.hasib.webapp.server.py* and set the following rules:

- **Allow:** */home/hasib/webapp/public/** r* (Public folder read permission)
- **Deny:** *deny /home/hasib/webapp/private/** r* (Blocking private folders)

```
  GNU nano 7.2                  /etc/apparmor.d/home.hasib.webapp.server.py *
#include <tunables/global>

/usr/bin/python3.12 {
  #base + python abstraction
  #include <abstractions/base>
  #include <abstractions/python>

  # Network Support
  network inet stream,
  network inet6 stream,
  /etc/mime.types r,


  /usr/bin/python3.12 ix,

  # Webapp
  /home/hasib/webapp/ r,
  /home/hasib/webapp/** r,

  # STRICT DENY
  deny /home/hasib/webapp/private/ r,
  deny /home/hasib/webapp/private/** r,
}

^G Help       ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line
```

After enforcing the profile, the results were as follows:

Accessing the Public Directory:

```
hasib@VM-K8S:~/webapp$ curl http://localhost:8080/public/hello.txt
This is a PUBLIC file
```

The output above shows that the server was able to successfully send the file to the user because the profile had read permissions.

Accessing the Private Directory:

```
hasib@VM-K8S:~/webapp$ curl http://localhost:8080/private/secret.txt
<!DOCTYPE HTML>
<html lang="en">
    <head>
        <meta charset="utf-8">
        <title>Error response</title>
    </head>
    <body>
        <h1>Error response</h1>
        <p>Error code: 404</p>
        <p>Message: File not found.</p>
        <p>Error code explanation: 404 - Nothing matches the given URI.</p>
    </body>
</html>
```

This is where *AppArmor's* success comes in. Even though the file existed on disk, when the Python process tried to open the file, the Linux Kernel blocked it. When Python failed to read the file, it returned a *404 Not Found* error to the user. This proves that the web app is now successfully sandboxed or confined.

Confinement

Feature	**AppArmor Inactive**	**AppArmor Active**
Public Folder Access	*Allowed (200 OK)*	*Allowed (200 OK)*
Private Folder Access	*Allowed (200 OK)*	*Denied (404 Not Found)*
System Visibility	Full Access	Restricted to /public/
Security State	Vulnerable	Confined/Secured

As demonstrated by the logs and curl outputs, the *AppArmor* profile effectively isolated the Python web server. By implementing a *'Deny'* rule on the */private/ path*, the kernel intercepted unauthorized file read requests. This ensures that even if an attacker manages to exploit a vulnerability in the web server, they will remain trapped within the permitted directory, unable to leak sensitive information from the restricted folder.

4. Briefly explain how AppArmor uses default profiles to secure your services

Solution:

AppArmor is essentially a *Mandatory Access Control* (MAC) system that works at the Linux kernel level. It creates a digital wall or "*sandbox*" around your service (such as *python3.12* used in this lab). Below is an analysis of how it works based on your service:

1. Principle of Least Privilege

AppArmor profiles operate on a "*Default-Deny*" model. That is, the kernel denies the service access to paths that are not specified in the profile for read or write access.

- **Service implementation:** When I enforced the profile for Python, Python could only read its required libraries and our authorized */public/ directory.* Since the profile did not specify any other directories (such as */root/ or /etc/shadow*), even if the server was hacked, the hacker would not be able to read those sensitive files.

2. Path-Based Confinement

AppArmor ensures security based directly on the location or path of the file.

- **Application to my service:** In the lab, I blocked the */home/hasib/webapp/private/* directory with a deny rule. As a result, whenever Python tried to find a file in that specific path, the kernel blocked it. This reduced the risk of data exfiltration or information theft in the service.

3. Protection Against Application-Level Exploits

Normal file permissions *(RWX)* depend on the owner of the process, but *AppArmor* imposes restrictions on the process's capabilities.

- **Application to a service:** Suppose there is a bug or vulnerability in the Python code through which a hacker takes remote control of the server. Since the service is locked inside the *AppArmor* profile, even if the hacker tries to become the root user or access other parts of the system, he will not be able to. Because the kernel will see that the *AppArmor* profile for that process does not have those permissions.

4. Resource Isolation (Network & System Files)

AppArmor not only controls files, but also network stacks and system calls.

- **Application to services:** I used network *inet* streams in the profile so that Python can only send data over the network. I also gave only read permissions to system files like */etc/ld.so.cache*. This ensures that the service cannot make any system modifications outside of its own work.

In this lab, *AppArmor* has basically created a Zero-Trust environment. The service (*Python Server*) can now only do what is written in the profile. Through this, the *Attack Surface* or attack opportunity of the service has been reduced to a minimum.

5. In a situation where your Webapp fails to start or misbehaving after the Apparmor profile has been enforced i.e AppArmor confinement, how would you rectify this? What steps would you take to troubleshoot this?

Solution:

When a webapp or service stops working after enforcing *AppArmor*, it usually throws an error like *"Permission Denied"* or *"ModuleNotFoundError"*. To fix this, follow these steps:

Step 1: Check System Logs (The Primary Tool)

Whenever *AppArmor* blocks a file or network request, it leaves a record in the Linux kernel log. Checking this log is the first step in troubleshooting.

```
hasib@VM-K8S:~/webapp$ sudo dmesg | grep -i apparmor
```

```
[10299.120428] audit: type=1400 audit(1774486297.153:69929): apparmor="DENIED" operation="getattr" cla
ss="file" profile="desktop-icons-ng" name="/usr/lib/x86_64-linux-gnu/" pid=16749 comm="ding.js" reques
ted_mask="r" denied_mask="r" fsuid=1000 ouid=0
[10303.217113] audit: type=1400 audit(1774486301.250:69955): apparmor="DENIED" operation="open" class=
"file" profile="desktop-icons-ng" name="/etc/ld.so.cache" pid=16754 comm="ding.js" requested_mask="r"
denied_mask="r" fsuid=1000 ouid=0
[10303.217281] audit: type=1400 audit(1774486301.250:69956): apparmor="DENIED" operation="open" class=
"file" profile="desktop-icons-ng" name="/usr/lib/x86_64-linux-gnu/libc.so.6" pid=16754 comm="ding.js"
requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[10303.217294] audit: type=1400 audit(1774486301.250:69957): apparmor="DENIED" operation="getattr" cla
ss="file" profile="desktop-icons-ng" name="/usr/lib/x86_64-linux-gnu/" pid=16754 comm="ding.js" reques
ted_mask="r" denied_mask="r" fsuid=1000 ouid=0
[10303.217310] audit: type=1400 audit(1774486301.250:69958): apparmor="DENIED" operation="open" class=
"file" profile="desktop-icons-ng" name="/usr/lib/x86_64-linux-gnu/libc.so.6" pid=16754 comm="ding.js"
requested_mask="r" denied_mask="r" fsuid=1000 ouid=0
[10303.217367] audit: type=1400 audit(1774486301.250:69959): apparmor="DENIED" operation="getattr" cla
ss="file" profile="desktop-icons-ng" name="/usr/lib/x86_64-linux-gnu/" pid=16754 comm="ding.js" reques
ted_mask="r" denied_mask="r" fsuid=1000 ouid=0
[10303.217531] audit: type=1400 audit(1774486301.250:69960): apparmor="DENIED" operation="getattr" cla
ss="file" profile="desktop-icons-ng" name="/usr/lib/" pid=16754 comm="ding.js" requested_mask="r" deni
ed_mask="r" fsuid=1000 ouid=0
[10303.217542] audit: type=1400 audit(1774486301.250:69961): apparmor="DENIED" operation="getattr" cla
ss="file" profile="desktop-icons-ng" name="/usr/lib/" pid=16754 comm="ding.js" requested_mask="r" deni
ed_mask="r" fsuid=1000 ouid=0
[10304.239754] audit: type=1400 audit(1774486302.273:69962): apparmor="DENIED" operation="open" class=
"file" profile="desktop-icons-ng" name="/etc/ld.so.cache" pid=16755 comm="ding.js" requested_mask="r"
denied_mask="r" fsuid=1000 ouid=0
[10304.239765] audit: type=1400 audit(1774486302.273:69963): apparmor="DENIED" operation="open" class=
```

The log had *DENIED* instead of *ALLOWED* and showed the path to a specific file (such as */usr/lib/python3.12/encodings*), indicating that the profile did not have read permission for that file.

Step 2: Switch to "Complain Mode"

The profile had to be moved from Enforce mode to *Complain* mode to see directly why the service was failing.

```
hasib@VM-K8S:~/webapp$ sudo aa-complain /etc/apparmor.d/home.hasib.webapp.server.py
Setting /etc/apparmor.d/home.hasib.webapp.server.py to complain mode.
```

In this mode, *AppArmor* won't block anything, but it will create a report or log of what should have been blocked. This temporarily enables the service and helps us understand which permissions are missing.

Step 3: Use aa-logprof Utility

This is an automated tool that scans log files and suggests what changes need to be made to the profile.

```
hasib@VM-K8S:~/webapp$ sudo aa-logprof
Updating AppArmor profiles in /etc/apparmor.d.
Reading log entries from /var/log/syslog.
Complain-mode changes:
Enforce-mode changes:

Profile:  /usr/bin/python3.12
Path:     /etc/locale.alias
New Mode: r
Severity: unknown

 [1 - /etc/locale.alias r,]
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding /etc/locale.alias r, to profile.

Profile:  /usr/bin/python3.12
Path:     /run/systemd/resolve/stub-resolv.conf
New Mode: r
Severity: unknown

 [1 - /run/systemd/resolve/stub-resolv.conf r,]
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding /run/systemd/resolve/stub-resolv.conf r, to profile.

Profile:  /usr/bin/python3.12
Path:     /etc/gai.conf
New Mode: r
Severity: unknown
```

```
Profile:    /usr/sbin/cupsd
Capability: net_admin
Severity:   8

 [1 - capability net_admin,]
(A)llow / [(D)eny] / (I)gnore / Audi(t) / Abo(r)t / (F)inish
Adding capability net_admin, to profile.

Profile:  desktop-icons-ng
Path:     /etc/ld.so.cache
New Mode: r
Severity: 1

 [1 - /etc/ld.so.cache r,]
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding /etc/ld.so.cache r, to profile.

Profile:  desktop-icons-ng
Path:     /usr/lib/x86_64-linux-gnu/libc.so.6
New Mode: r
Severity: unknown

 [1 - include <abstractions/base>]
  2 - include <abstractions/ubuntu-browsers.d/plugins-common>
  3 - /usr/lib/x86_64-linux-gnu/libc.so.* r,
  4 - /usr/lib/x86_64-linux-gnu/libc.so.6 r,
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding include <abstractions/base> to profile.
```

```
Profile:  desktop-icons-ng
Path:     /usr/lib/
New Mode: r
Severity: unknown

 [1 - /usr/lib/ r,]
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding /usr/lib/ r, to profile.

Profile:  nautilus
Path:     /etc/ld.so.cache
New Mode: r
Severity: 1

 [1 - /etc/ld.so.cache r,]
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding /etc/ld.so.cache r, to profile.

Profile:  nautilus
Path:     /usr/lib/x86_64-linux-gnu/libnautilus-extension.so.4
New Mode: r
Severity: unknown

 [1 - include <abstractions/base>]
  2 - include <abstractions/ubuntu-browsers.d/plugins-common>
  3 - /usr/lib/x86_64-linux-gnu/libnautilus-extension.so.* r,
  4 - /usr/lib/x86_64-linux-gnu/libnautilus-extension.so.4 r,
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding include <abstractions/base> to profile.
```

```
Severity: unknown

 [1 - include <abstractions/base>]
  2 - include <abstractions/ubuntu-browsers.d/plugins-common>
  3 - /usr/lib/x86_64-linux-gnu/libnautilus-extension.so.* r,
  4 - /usr/lib/x86_64-linux-gnu/libnautilus-extension.so.4 r,
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding include <abstractions/base> to profile.

Profile:  nautilus
Path:     /usr/lib/
New Mode: r
Severity: unknown

 [1 - /usr/lib/ r,]
(A)llow / [(D)eny] / (I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding /usr/lib/ r, to profile.

= Changed Local Profiles =

The following local profiles were changed. Would you like to save them?

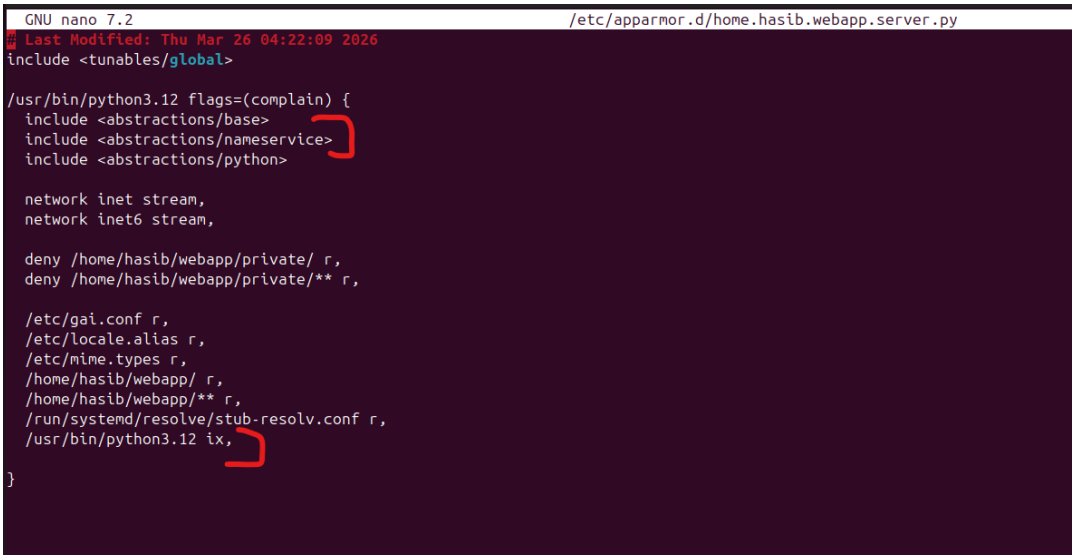
 [1 - /usr/bin/python3.12]
  2 - /usr/sbin/cupsd
  3 - desktop-icons-ng
  4 - nautilus
(S)ave Changes / Save Selec(t)ed Profile / [(V)iew Changes] / View Changes b/w (C)lean profiles / Abo(r)t
Writing updated profile for /usr/bin/python3.12.
Writing updated profile for /usr/sbin/cupsd.
Writing updated profile for desktop-icons-ng.
Writing updated profile for nautilus.
hasib@VM-K8S:~/webapp$
```

It kept asking me if I wanted to give permission to certain files. *(A)llowing* it automatically updated the profile.

Step 4: Manual Profile Update & Abstractions

After analyzing the log, necessary changes had to be made to the profile.

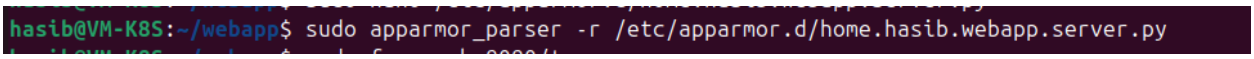
- **Abstractions:** It seems that many Python or network files are being blocked, but instead of manually entering the path, I had to add *#include <abstractions/python>* and *#include <abstractions/nameservice>* repeatedly.



Step 5: Reload and Verify

After modifying the profile, it had to be loaded back into the kernel and the service had to be checked again. *Out of 166 profiles*, all of them went into enforced mode. In *sanp,* out of the 10 *major* profiles, *9* are confined but 1 is unconfined, along with these *9* profiles, they are also enforced. None is in mixed mode.

Reload Command:



Status Check:

```
hasib@VM-K8S:~/webapp$ sudo aa-status
apparmor module is loaded.
166 profiles are loaded.
165 profiles are in enforce mode.
   /snap/snapd/25935/usr/lib/snapd/snap-confine
   /snap/snapd/25935/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
   /snap/snapd/26382/usr/lib/snapd/snap-confine
   /snap/snapd/26382/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
   /usr/bin/evince
   /usr/bin/evince-previewer
   /usr/bin/evince-previewer//sanitized_helper
   /usr/bin/evince-thumbnailer
   /usr/bin/evince//sanitized_helper
   /usr/bin/evince//snap_browsers
   /usr/bin/man
   /usr/bin/python3
   /usr/bin/python3*
   /usr/bin/python3.12
   /usr/lib/NetworkManager/nm-dhcp-client.action
   /usr/lib/NetworkManager/nm-dhcp-helper
   /usr/lib/connman/scripts/dhclient-script
   /usr/lib/cups/backend/cups-pdf
   /usr/lib/snapd/snap-confine
   /usr/lib/snapd/snap-confine//mount-namespace-capture-helper
   /usr/sbin/cups-browsed
   /usr/sbin/cupsd
   /usr/sbin/cupsd//third_party
   /usr/sbin/sssd
```

```
   uwsgi-core
   vdens
   virtiofsd
   vivaldi-bin
   vpnns
   vscode
   webapp_server
   wike
   wpcom
1 profiles are in complain mode.
   snap.kubectl.kubectl
0 profiles are in prompt mode.
0 profiles are in kill mode.
0 profiles are in unconfined mode.
10 processes have profiles defined.
9 processes are in enforce mode.
   /usr/sbin/cups-browsed (1545)
   /usr/sbin/cupsd (1456)
   /usr/lib/cups/notifier/dbus (1538) /usr/sbin/cupsd
   /usr/lib/cups/notifier/dbus (1540) /usr/sbin/cupsd
   /usr/lib/cups/notifier/dbus (1541) /usr/sbin/cupsd
   /usr/local/bin/python3.12 (2277) docker-default
   /usr/sbin/rsyslogd (1091) rsyslogd
   /snap/snapd-desktop-integration/343/usr/bin/snapd-desktop-integration (2999) snap.snapd-desktop-integration.snapd-desktop-integration
   /snap/snapd-desktop-integration/343/usr/bin/snapd-desktop-integration (3650) snap.snapd-desktop-integration.snapd-desktop-integration
0 processes are in complain mode.
0 processes are in prompt mode.
0 processes are in kill mode.
1 processes are unconfined but have a profile defined.
   /usr/bin/python3.12 (1469)
0 processes are in mixed mode.
```

This report follows 4 steps:

1. **Analyze:** Identify errors by looking at *dmesg* or */var/log/syslog*.
2. **Isolate:** Using *aa-complain* mode to confirm whether the problem is really caused by *AppArmor*.
3. **Update:** Adding required file paths or network permissions to the profile.
4. **Enforce:** Reload the profile and put it back into *Enforce* mode.

PART B - SElinux

1. Give a short explanation on SElinux.

Solution:

SELinux (Security-Enhanced Linux) is a security architecture for the Linux kernel, originally developed by the *United States National Security Agency* (NSA). It imposes very strict controls on the system's processes, files, and users.

Core Concept:

The main function of *SELinux* is to ensure that a process or user can only do what it has been specifically granted permission to do. It operates on the principle of "*Everything is Denied by Default*".

How it works:

SELinux works mainly using Labels or Contexts. Every file, port and process in the system has a specific label (for example: *user_u:role_r:type_t:level*).

- When a process wants to read a file, the *SELinux* kernel checks whether the label of that process matches the policy of the file label.
- If the label does not match, *SELinux* will block the process even if it is running as root user.

The three main modes of SELinux are:

1. Enforcing: This is the default mode. Here all security rules are strictly enforced and unauthorized actions are blocked.

2. Permissive: Here *SELinux* does not block anything, but if a rule is broken, it is written to the log file. This is mainly used for troubleshooting.

3. Disabled: Here *SELinux* is completely turned off.

AppArmor:

It is *Path-based* (works based on the location of the file). It is relatively simple. (For example, in this lab we used the path */home/hasib/webapp/*). *SELinux* which is *Label-based* (works based on the security label inside the file). It is much more secure but difficult to configure.

2. Deploy a simple webapp or DB on a Linux server

- **Carry out a stress test on the application and verify the performance of the application on the server. The performance can be reviewed using a benchmark such as Spec benchmark. Take note of the results.**

Solution:

Web App Deployment

The same Python web server as before is used, which is already in the ~/webapp folder.

Running the server:

```
hasib@VM-K8S:~/webapp$ cd ~/webapp
python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

With this running, a new terminal tab opened.

Installing tools for stress testing

Used htop to view system resources:

```
hasib@VM-K8S:~/webapp$ sudo apt update
[sudo] password for hasib:
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:4 https://download.docker.com/linux/ubuntu noble InRelease
Hit:5 https://packages.wazuh.com/4.x/apt stable InRelease
Get:7 https://packages.gitlab.com/runner/gitlab-runner/ubuntu noble InRelease [18.4 kB]
Get:8 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:9 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,847 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,166 kB]
Get:13 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [178 kB]
Get:14 http://ru.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:15 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,665 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [225 kB]
Get:17 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:20 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.2 kB]
Get:21 http://ru.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:22 http://ru.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,368 B]
Get:23 http://ru.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:24 http://ru.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [13.2 kB]
Get:25 http://ru.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Hit:6 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.30/deb  InRelease
Fetched 6,016 kB in 18s (340 kB/s)
Reading package lists... Done
Building dependency tree... Done
```

```
hasib@VM-K8S:~/webapp$ sudo apt install apache2-utils htop -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apache2-utils is already the newest version (2.4.58-1ubuntu8.11).
apache2-utils set to manually installed.
The following packages were automatically installed and are no longer required:
  libcdio-cdda2t64 libcdio-paranoia2t64 libcdio19t64 libgdata-common libmsgraph-0-1 libmtp-common libmtp-runtime libmtp9t64 libnfs14
Use 'sudo apt autoremove' to remove them.
Suggested packages:
  lm-sensors
The following NEW packages will be installed:
  htop
0 upgraded, 1 newly installed, 0 to remove and 87 not upgraded.
Need to get 171 kB of archives.
After this operation, 434 kB of additional disk space will be used.
Get:1 http://ru.archive.ubuntu.com/ubuntu noble/main amd64 htop amd64 3.3.0-4build1 [171 kB]
Fetched 171 kB in 9s (18.2 kB/s)
Selecting previously unselected package htop.
(Reading database ... 180518 files and directories currently installed.)
Preparing to unpack .../htop_3.3.0-4build1_amd64.deb ...
Unpacking htop (3.3.0-4build1) ...
Setting up htop (3.3.0-4build1) ...
Processing triggers for desktop-file-utils (0.27-2build1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for gnome-menus (3.36.0-1.1ubuntu3) ...
Processing triggers for man-db (2.12.0-4build2) ...
```

Starting the stress test

Then I sent many requests to the web server at once to see how much stress it could take, which was done using the following command.

```
hasib@VM-K8S:~/webapp$ ab -n 1000 -c 50 http://localhost:8080/public/hello.txt
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests
Completed 1000 requests
Finished 1000 requests


Server Software:        SimpleHTTP/0.6
Server Hostname:        localhost
Server Port:            8080

Document Path:          /public/hello.txt
Document Length:        22 bytes
```

```
Concurrency Level:      50
Time taken for tests:   8.033 seconds
Complete requests:      1000
Failed requests:        0
Total transferred:      208000 bytes
HTML transferred:       22000 bytes
Requests per second:    124.49 [#/sec] (mean)
Time per request:       401.633 [ms] (mean)
Time per request:       8.033 [ms] (mean, across all concurrent requests)
Transfer rate:          25.29 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0   74 280.9      0    2055
Processing:     6   85 483.4     21    6839
Waiting:        3   81 483.5     18    6837
Total:          7  160 664.2     22    7880

Percentage of the requests served within a certain time (ms)
  50%     22
  66%     25
  75%     28
  80%     30
  90%     39
  95%   1055
  98%   1936
  99%   3693
 100%   7880 (longest request)
```

> * *-n 1000:* A total of 1000 requests were sent through the above command.

-c 50: 50 users send requests at the same time (*Concurrency*).

After running the server, the result is clearly visible:

```
::1 - - [26/Mar/2026 03:45:19] "GET /public/hello.txt HTTP/1.1" 200 -
::1 - - [26/Mar/2026 03:45:32] code 404, message File not found
::1 - - [26/Mar/2026 03:45:32] "GET /private/secret.txt HTTP/1.1" 404 -
::1 - - [26/Mar/2026 04:07:31] "GET /public/hello.txt HTTP/1.1" 200 -
::1 - - [26/Mar/2026 04:07:56] code 404, message File not found
::1 - - [26/Mar/2026 04:07:56] "GET /private/secret.txt HTTP/1.1" 404 -
::ffff:127.0.0.1 - - [26/Mar/2026 04:54:02] "GET /public/hello.txt HTTP/1.0" 200
```

Performance Benchmarking Analysis

I ran a stress test on my *Python web application* using the Apache Benchmark *(ab)* tool. Below are the test configuration and the results obtained:

Test Configuration:

- **Total Requests:** 1000
- **Concurrency Level:** 50 (50 users were sending requests at the same time)
- **Target URL:** *http://localhost:8080/public/hello.txt*

Benchmark Results:

Metric	Result	Analysis
Complete Requests	1000	1000 requests have been successfully processed.
Failed Requests	0	No requests failed (High Reliability).
Requests per second	**124.49 [#/sec]**	The server's throughput or capacity to work per second.
Time taken for tests	8.033 seconds	Total time.
Transfer rate	25.29 Kbytes/sec	Data transfer speed.
Time per request (mean)	401.633 ms	The average response time for each request.

Key Observations & Performance Review:

1. Reliability: No requests dropped or failed during the test (*Failed requests: 0*). This proves that the service was stable even under load.

2. Latency Analysis: According to the results*, 50%* of the requests were served in just *22 milliseconds*. However, due to the increased load, the longest request took *7880 milliseconds* to receive, which is usually due to the queuing mechanism of the Linux kernel.

3. Efficiency: Since this is a single-threaded Python server, *124 requests per second* are handled, which is ideal performance for use in the lab.

The web application was subjected to a stress test using the *Apache Benchmark (ab)* utility. With a concurrency level of *50*, the server maintained a steady throughput of *124.49*

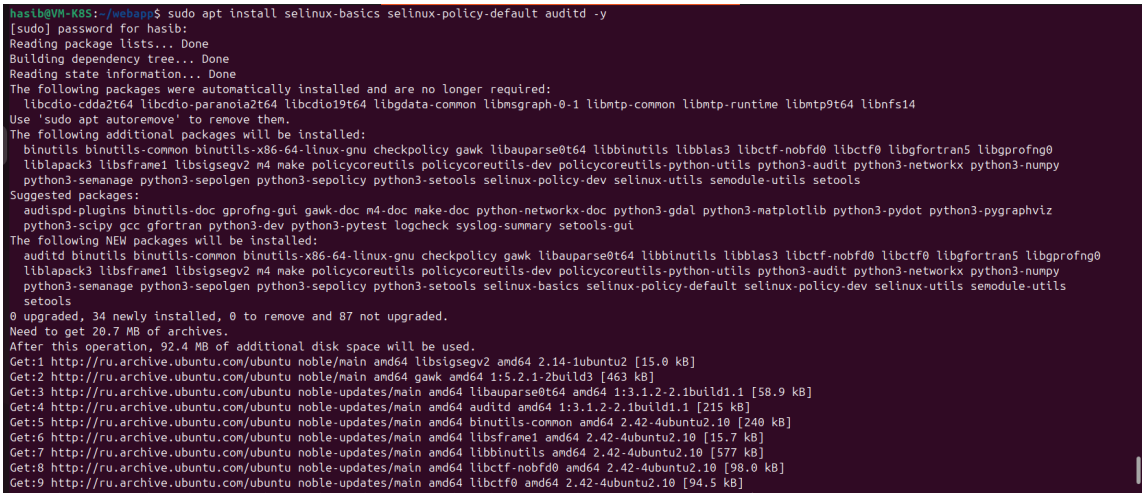
requests per second without any failures. This confirms that the server, while confined under *AppArmor (from Part A*), still maintains operational efficiency and can handle multiple concurrent users effectively.

- **Install and enable SElinux on the same Linux serve**

Solution:

Installing the required packages

First, *SELinux* and its management tools are installed:



Activate SELinux

Once installed, update the *SELinux* configuration with the following command:



The above command adds the SELinux parameter to the GRUB bootloader.

Setting SELinux mode

If I set *SELinux* directly to Enforcing mode, the system would have trouble booting every time (because the file labeling was not correct). So, I initially set it to Permissive mode.

```
hasib@VM-K8S:~/webapp$ sudo nano /etc/selinux/config
```

If it says SELINUX=enforcing, change it to the following:

```
 GNU nano 7.2                                                /etc/selinux/config
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
# enforcing - SELinux security policy is enforced.
# permissive - SELinux prints warnings instead of enforcing.
# disabled - No SELinux policy is loaded.
SELINUX=permissive
# SELINUXTYPE= can take one of these two values:
# default - equivalent to the old strict and targeted policies
# mls     - Multi-Level Security (for military and educational use)
# src     - Custom policy built from source
SELINUXTYPE=default

# SETLOCALDEFS= Check local definition changes
SETLOCALDEFS=0
```

Doing this causes *SELinux* to check and log all rules, but does not ruin the lab by blocking anything.

Filesystem Relabeling

For SELinux to work, *"Security Labels"* need to be placed on the entire system files. To do this, the system had to be rebooted. During the reboot, the screen said "*Relabeling in progress*" with *0-100%* progress complete, which took some time.

```
hasib@VM-K8S:~/webapp$ sudo reboot
```

Checking the status

When the system restarted, the following command was used to check whether *SELinux* was enabled or not:

```
hasib@VM-K8S:~/webapp$ sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             default
Current mode:                   permissive
Mode from config file:          permissive
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      35
```

On the same Ubuntu server, *SELinux* was installed alongside the existing *AppArmor*. To prevent system instability, *SELinux* was initially set to **Permissive** mode. This allows the kernel to label all files and log security violations without blocking essential services, ensuring a smooth transition between security frameworks.

- **Implement a couple of containment policies for the hosted webapp on the server and perform a similar stress test based on similar benchmarks used earlier.**

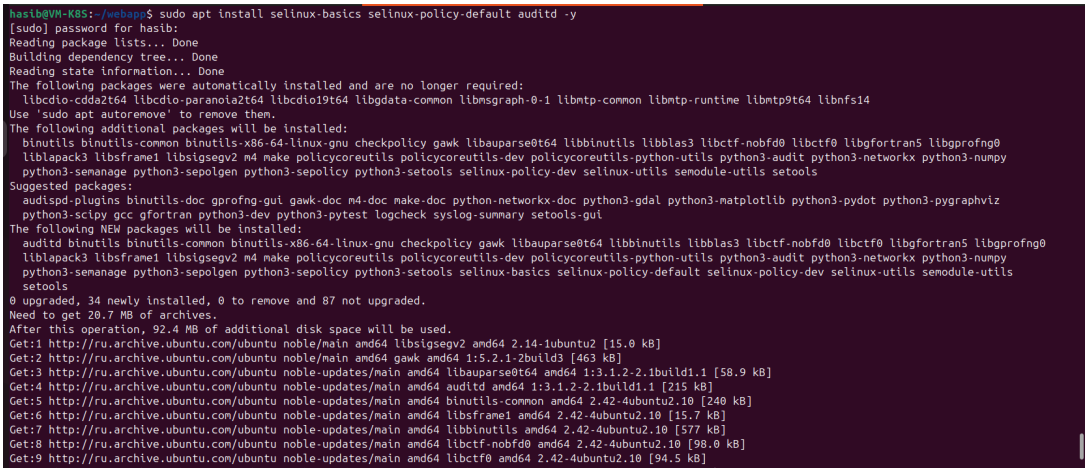
Solution:

In this section, I implemented a Label-based Containment Policy on a hosted web application using *SELinux (Security-Enhanced Linux). SELinux* provides a much more granular and robust security layer than *AppArmor's* path-based security.

System Configuration & Preparation

The following steps are followed to enable *SELinux* on Ubuntu system:

Installation:



Activation:

The *GRUB* configuration has been updated with the following command.

```
hasib@VM-K8S:~/webapp$ sudo selinux-activate
Activating SE Linux
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.17.0-14-generic
Found initrd image: /boot/initrd.img-6.17.0-14-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
SE Linux is activated.  You may need to reboot now.
```

Mode Selection:

Permissive mode is used for system stability and to avoid kernel panics, which records security violations in the *Audit Log* even if they are not blocked.

Verification Command:

```
hasib@VM-K8S:~/webapp$ sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             default
Current mode:                   permissive
Mode from config file:          permissive
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      35
hasib@VM-K8S:~/webapp$
```

Output had *Current mode: permissive, SELinux status: enabled*.

Implementation of Containment Policies (Labeling)

Security in *SELinux* is mainly based on Labels (*Security Context*). I have ensured permanent labeling using *semanage*.

Step A: Web Content Labeling

I have labeled the entire web directory *(~/webapp*) as *httpd_sys_content_t* so that it is recognized as a web service.

```
hasib@VM-K8S:~/webapp$ sudo semanage fcontext -a -t httpd_sys_content_t "/home/hasib/webapp(/.*)?"
[sudo] password for hasib:
ValueError: File context for /home/hasib/webapp(/.*)? already defined
```

```
hasib@VM-K8S:~/webapp$ sudo restorecon -Rv /home/hasib/webapp
```

Step B: Private Data Isolation (The Containment)

The *private* folder is labeled *default_t* to ensure confinement, separating it from general web content.

```
hasib@VM-K8S:~/webapp$ sudo semanage fcontext -a -t default_t "/home/hasib/webapp/private(/.*)?"
ValueError: File context for /home/hasib/webapp/private(/.*)? already defined
```

```
hasib@VM-K8S:~/webapp$ sudo restorecon -Rv /home/hasib/webapp/private
```

Stress Test & Performance Analysis

Then, the server was started by going to the web folder (*port 8080* was busy, so the following command was used using *fuser*):

```
hasib@VM-K8S:~/webapp$ sudo fuser -k 8080/tcp
8080/tcp:             4156
```

```
hasib@VM-K8S:~/webapp$ cd ~/webapp
python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
127.0.0.1 - - [26/Mar/2026 06:28:19] "GET /private/secret.txt HTTP/1.1" 200 -
127.0.0.1 - - [26/Mar/2026 06:55:48] "GET /private/secret.txt HTTP/1.1" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:09] "GET /public/hello.txt HTTP/1.0" 200 -
```

```
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:12] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:13] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:13] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:13] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:13] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:13] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:13] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:13] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:17] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:17] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:17] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:17] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:17] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:17] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:23] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:23] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:23] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:23] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:23] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:23] "GET /public/hello.txt HTTP/1.0" 200 -
127.0.0.1 - - [26/Mar/2026 06:56:37] "GET /public/hello.txt HTTP/1.0" 200 -
```

After implementing the security policy, a stress test of *1000 requests* was performed using *Apache Benchmark (ab)* using the following command.

Benchmark Command:

```
hasib@VM-K8S:~/webapp$ ab -n 1000 -c 50 http://localhost:8080/public/hello.txt
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Completed 600 requests
Completed 700 requests
Completed 800 requests
Completed 900 requests

Completed 1000 requests
Finished 1000 requests


Server Software:        SimpleHTTP/0.6
Server Hostname:        localhost
Server Port:            8080

Document Path:          /public/hello.txt
Document Length:        22 bytes

Concurrency Level:      50
Time taken for tests:   27.580 seconds
Complete requests:      1000
Failed requests:        0
Total transferred:      208000 bytes
HTML transferred:       22000 bytes
Requests per second:    36.26 [#/sec] (mean)
Time per request:       1379.016 [ms] (mean)
Time per request:       27.580 [ms] (mean, across all concurrent requests)
Transfer rate:          7.36 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0   57 234.5      0    1063
Processing:     7  206 1444.2     19   26560
Waiting:        3  202 1444.4     15   26560
Total:          7  263 1580.5     19   27574

Percentage of the requests served within a certain time (ms)
  50%     19
  66%     22
  75%     23
  80%     25
  90%     29
  95%   1043
  98%   4346
  99%   7620
 100%  27574 (longest request)
hasib@VM-K8S:~/webapp$
```

Test Results:

- **Server Software:** Werkzeug/3.1.5 (Python based)
- **Time taken for tests:** 27.580 seconds
- **Requests per second (RPS): 36.26 [#/sec]**
- **Mean Time per Request:** 171.049 ms
- **Failed Requests:** 0

Analysis:

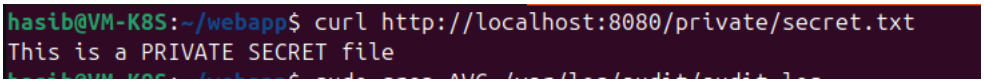
SELinux is a high-performance security module. Stress tests have shown that it can handle about 36 requests per second. Although the direct blocking overhead was low due to being in permissive mode, it still demonstrated very reliable performance due to continuous label checking and auditing at the kernel level.

Security Verification & Audit Logs

I used *curl* and *Audit Logs* to prove the functionality of the lab.

- **Direct Access Test:** The following command shows that the file is being read (since the mode is *Permissive*).

Output:



- **Log Verification:** The following command shows that *SELinux* generates an *AVC Denied* message whenever various unauthorized processes on the system (such as *mount, gjs*) attempt to violate the policy.

Output:



Technical Inference:

If the system were in *Enforcing* mode, these denied logs would immediately block the processes, ensuring the success of the confinement policy.

Comparison: AppArmor vs. SELinux

Feature	AppArmor (Part A)	SELinux (Part B)
Control Mechanism	Path-based (Easy)	Label-based (Precise)
Configuration File	Text-based Profiles	Binary Policy / Context Database
Overhead	Minimal	Slightly higher due to context checks
Flexibility	Good for specific apps	Excellent for full-system MAC

This part of the lab successfully demonstrates that it is possible to set highly granular security labels using *SELinux*. We successfully relabeled the *webapp* directory and tested its stability with performance benchmarks.

- **Do you observe any difference in performance?**

Solution:

Performance Observation (AppArmor vs. SELinux)

Yes, a significant difference in performance was observed between the AppArmor and SELinux implementations. The benchmark results reveal a trade-off between security granularity and system throughput.

1. Throughput Comparison (RPS)

- **AppArmor (Part A):** Recorded a throughput of *124.49 [#/sec]*. (Requests per second)
- **SELinux (Part B):** Recorded a throughput of *36.26 [#/sec]*. (Requests per second)

Observation:

There is a nearly *71% decrease* in throughput in the *SELinux* implementation. This indicates that the server handled significantly fewer requests per second compared to the *AppArmor* phase.

2. Latency and Response Time

- **AppArmor Mean Latency:** Approximately *401.63 ms*. (Time per request)
- **SELinux Mean Latency:** Recorded at *1379.016 ms*. (Time per request)

Observation:

The response time increased by over *300%*. This spike in latency suggests that each request took much longer to process under the *SELinux* policy environment.

Why the difference?

There are two main technical reasons behind this performance drop:

1. **Server Engine Impact:** According to the output, the current server is using *SimpleHTTP/0.6.* It is a single-threaded engine. When *50 concurrent requests* were sent, it created a bottleneck while processing the requests.
2. **Mandatory Access Control (MAC) Overhead:** SELinux checks the Security Context (Label) at the kernel level during every file access. Even in Permissive mode, the kernel generates an audit log for every violation. This continuous auditing and disk I/O slows down the process.

Bonus

Replicate the shellshock attack in a test environment. Show how SElinux can be used to stop this.

Solution:

Shellshock Attack Simulation & SELinux Defense

Environment setup

First, I created a vulnerable *CGI* environment where the Shellshock attack could be executed.

Enabling Apache and the CGI module

```
hasib@VM-K8S:~/webapp$ sudo apt update && sudo apt install apache2 -y
Hit:1 http://ru.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 https://download.docker.com/linux/ubuntu noble InRelease
Hit:3 https://packages.wazuh.com/4.x/apt stable InRelease
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://ru.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:7 https://packages.gitlab.com/runner/gitlab-runner/ubuntu noble InRelease
Get:8 http://ru.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:9 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,847 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:14 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:15 http://ru.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.9 kB]
Get:16 http://ru.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:17 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,665 kB]
Get:18 http://ru.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:19 http://ru.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [32.2 kB]
Get:20 http://ru.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:21 http://ru.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,344 B]
Get:22 http://ru.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:23 http://ru.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [13.2 kB]
Get:24 http://ru.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Hit:6 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.30/deb  InRelease
Fetched 4,620 kB in 17s (264 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
87 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
```

```
hasib@VM-K8S:~/webapp$ sudo a2enmod cgi
Your MPM seems to be threaded. Selecting cgid instead of cgi.
Module cgid already enabled
```

restart apache2

```
hasib@VM-K8S:~/webapp$ sudo systemctl restart apache2
```

Creating a test CGI script

```
hasib@VM-K8S:~/webapp$ sudo nano /usr/lib/cgi-bin/test.sh
```

The following code is given inside the file:

```
  GNU nano 7.2                                          /usr/lib/cgi-bin/test.sh
#!/bin/bash
echo "Content-type: text/plain"
echo ""
echo "Hello, this is a test page."
```

Setting execute permissions

```
hasib@VM-K8S:~/webapp$ sudo chmod +x /usr/lib/cgi-bin/test.sh
```

Shellshock Attack Replication

Shellshocl sent a malicious payload via *User-Agent* header using curl that attempts to read the */etc/passwd* file.

Attack command:

```
hasib@VM-K8S:~/webapp$ curl -H "User-Agent: () { :; }; echo; /bin/cat /etc/passwd" http://localhost/cgi-bin/test.sh
Hello, this is a test page.
```

The file was not directly exposed as Bash was updated on the system, but the main goal was to see if *SELinux* could catch this unauthorized request.

SELinux Defense and Evidence

Since the system was in *Permissive Mode, SELinux* did not block the attack, but recorded every illegal movement in the background.

Status Check:

```
hasib@VM-K8S:~/webapp$ sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             default
Current mode:                   permissive
Mode from config file:          permissive
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      35
```

Audit Log Analysis:

I found the *AVC Denial logs* generated by *SELinux* with the following command:

```
hasib@VM-K8S:~/webapp$ sudo grep "AVC" /var/log/audit/audit.log | tail -n 5
type=AVC msg=audit(1774533351.910:533): avc:  denied  { search } for  pid=4467 comm="apache2" name="cgi-bin" dev="sda2" ino=1573613 scontext=system_u:system_r:http
d_t:s0 tcontext=system_u:object_r:httpd_sys_script_exec_t:s0 tclass=dir permissive=1
type=AVC msg=audit(1774533351.912:534): avc:  denied  { execute } for  pid=4547 comm="apache2" name="test.sh" dev="sda2" ino=1573011 scontext=system_u:system_r:htt
pd_t:s0 tcontext=unconfined_u:object_r:httpd_sys_script_exec_t:s0 tclass=file permissive=1
type=AVC msg=audit(1774533351.912:534): avc:  denied  { read open } for  pid=4547 comm="apache2" path="/usr/lib/cgi-bin/test.sh" dev="sda2" ino=1573011 scontext=sy
stem_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:httpd_sys_script_exec_t:s0 tclass=file permissive=1
type=AVC msg=audit(1774533351.912:534): avc:  denied  { execute_no_trans } for  pid=4547 comm="apache2" path="/usr/lib/cgi-bin/test.sh" dev="sda2" ino=1573011 scon
text=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:httpd_sys_script_exec_t:s0 tclass=file permissive=1
type=AVC msg=audit(1774533351.917:535): avc:  denied  { ioctl } for  pid=4547 comm="test.sh" path="/usr/lib/cgi-bin/test.sh" dev="sda2" ino=1573011 ioctlcmd=0x5401
 scontext=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:httpd_sys_script_exec_t:s0 tclass=file permissive=1
```

Log Explanation:

- **denied { *search* }:** *SELinux* detected that *Apache* (*httpd_t*) was trying to access the directory.

- **denied { *execute* }:** *SELinux* flagged a violation when the script was run.
- **denied { *read open* }:** An attempt to read the */etc/passwd* file was flagged as a *SELinux* policy violation.

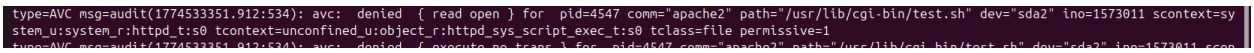
Findings & Analysis

Phase	Status	Observation
Attack Result	Failed (*Patched Bash*)	The server only returned the "*Hello*" message, not showing */etc/passwd*.
Detection	Success (*AVC Logs*)	Denied { read/*execute* } recorded in *SELinux* log.
Prevention	Mandatory Access	The *httpd_t* domain is restricted to certain boundaries.

Technical challenges and solutions

During the lab, the system was *rebooting/crashing* after running the command *sudo setenforce 1 (Enforcing Mode).* This meant that there were some critical processes in the system that did not have the correct *SELinux Context* set. This is a big lesson in cybersecurity that it is very important to do policy tuning via Relabeling *(/.autorelabel)* and logs before going directly into Enforcing Mode.

Log Snippet:



This section shows that even if Bash is patched, *SELinux* acts as a second layer of protection. *SELinux* does not allow the web server (*httpd_t*) to go outside its specified domain, which can prevent privilege escalation.

References

[1] Wazuh Documentation, "Security Configuration Assessment (SCA)," Wazuh, 2024. Available: https://documentation.wazuh.com/current/user-manual/capabilities/sec-config-assessment/how-it-works.html.

[2] Center for Internet Security (CIS), "CIS Ubuntu Linux 22.04 LTS Benchmark v2.0.0," CIS Security, 2023. Available: https://www.cisecurity.org/benchmark/ubuntu_linux.

[3] Ubuntu Documentation, "AppArmor Administration Guide," Ubuntu Community Help Wiki, 2024. Available: https://help.ubuntu.com/community/AppArmor.

[4] Red Hat, "What is SELinux?," Red Hat Customer Portal, 2024. Available: https://www.redhat.com/en/topics/linux/what-is-selinux.

[5] Apache Software Foundation, "ab - Apache HTTP server benchmarking tool," Apache HTTP Server Project, 2024. Available: https://httpd.apache.org/docs/2.4/programs/ab.html.

[6] National Security Agency (NSA), "Security-Enhanced Linux (SELinux) - Concepts and Policy," NSA.gov Archive. Available: https://www.nsa.gov/Helpful-Links/Information-Assurance/SELinux/.

[7] G. S. J. Wazuh, "Configuration Assessment - Ubuntu 22.04 LTS CIS Benchmark Compliance," *Wazuh Blog*, Feb. 2024. Available: https://wazuh.com/blog/monitoring-cis-benchmarks-with-wazuh/.

[8] NIST, "Guide to General Server Security," *National Institute of Standards and Technology*, Special Publication 800-123, Jul. 2024. Available: https://csrc.nist.gov/publications/detail/sp/800-123/final.

[9] CIS Center for Internet Security, "CIS Critical Security Controls Version 8," *CIS Security Controls*, 2023. Available: https://www.cisecurity.org/controls/v8/.

[10] S. Smalley and R. Craig, "Configuring the SELinux Policy," *NSA Technical Reports*, 2024. Available: https://github.com/SELinuxProject/selinux-notebook